HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Project Report – Embedded Systems and IoT Design - CC04

SMART PLANT MONITORING SYSTEM

USING ESP32 MICROCONTROLLER

- Group 5 -

Lecturer: Nguyễn Thiên Ân

Member Contribution

Tên	**MSSV**	**Task Given**	**Task Done**	**Contr. %**
Nguyễn Lê Cao Tướng	2353300	Hardware, Circuit Design & ESP32 Firmware	Designed the full wiring diagram & built and tested the physical circuit on breadboard. - **80%** Wrote the complete ESP32 firmware, including sensor reading, OLED output, EEPROM storage, and threshold logic. - **90%** Implemented MQTT publishing for all sensor topics and MQTT subscription for encrypted commands. - **100%** Integrated SIMON lightweight encryption into the ESP32 side, ensuring encrypted command handling matches the Python client. - **30%** Created the command-handling system for dynamic threshold updates. - **100%** Wrote, edited, and formatted the full project report. - **30%**	**35%**
Phạm Nhật Khôi	2352623	SIMON lightweight encryption	Researched SIMON lightweight block cipher and its key schedule. - **50%** Implemented SIMON 64/128 encryption and decryption on ESP32 in C/C++ & implemented matching SIMON encryption/decryption in Python for the PC client. - **40%** Wrote, edited, and formatted the full project report. - **30%**	**15%**

Trần Nguyễn Minh Khôi	2352625	Documentation, Team Support	Designed the full wiring diagram & built and tested the physical circuit on breadboard. - **20%** Wrote the ESP32 firmware for sensor reading and MQTT publishing. - **10%** Researched SIMON lightweight block cipher and its key schedule. - **50%** Wrote, edited, and formatted the full project report. - **40%**	**15%**
Nguyễn Việt	2353320	Python Application & MQTT Communication	Developed the Python program that receives encrypted sensor data, prints it clearly, and sends encrypted commands back to ESP32. - **100%** Implemented SIMON encryption/decryption in Python so messages match exactly with the ESP32 algorithm. - **30%** Set up MQTT connections, callbacks, and user input interface for sending thresholds or configuration values. - **100%** Tested communication reliability between Python and ESP32 using HiveMQ public broker. - **100%** Helped in verifying that encrypted commands get correctly decrypted and executed by the device. - **100%**	**35%**

Contents

Chapter 1

Introduction

1.1 Background

The Smart Plant Monitoring System is an in-built IoT system designed to automate plant development condition monitoring and analysis. The system incorporates several environmental sensors and an OLED display to gather, process, and show data in real-time using an ESP32 microcontroller as the main processing unit. Additionally, it offers a low-latency, effective connection with the main server or user application by wirelessly transmitting data using the MQTT protocol for remote monitoring.

This project uses the SIMON Lightweight Block Cipher, which encrypts data before wireless transmission and protects sensor readings from inappropriate interception or alteration, to improve security in Internet of Everything connections. As a result, the system provides a solid basis for future smart agriculture systems by combining environmental sensing, data display, secure connectivity, and configurability.

1.2 Project Objectives

This project's primary goals are to:

- Create and implement an ESP32-based compact embedded system for monitoring plant environmental conditions.
- Collect data in real time by integrating sensors such as DHT22, BH1750, MQ135, and a soil moisture sensor. Readings can be remotely transmitted via the MQTT protocol and displayed on a 0.96-inch OLED screen. For secure communication between the ESP32 and the MQTT broker, use SIMON encryption.
- Provide network-based commands and a serial interface for user interaction and system configuration.
- Demonstrate a modular architecture that allows for future expansion and integration into a complete IoT system.
- Ensure data retention by storing configurable thresholds in EEPROM.

1.3 Significance of the Project

This project exhibits actual knowledge of IoT systems, integrated software development, and secure data transmission, all of which are required capabilities in modern computer engineering. Automation and secure remote monitoring are essential components of sustainable agricultural practices, and they offer an educational foundation for the development of smart agriculture technology.

From an engineering standpoint, the Smart Plant Monitoring System demonstrates how communication protocols, firmware programming, hardware design, and lightweight cryptography can all be integrated into a single embedded platform.

Chapter 2

System Overview

2.1 System Concept

The Smart Plant Monitoring System is designed as a self-contained environmental data station. Using sensors, it collects environmental information in real-time and provides both visual and textual outputs to the user. The system architecture emphasizes modularity, where each sensor or function can be independently upgraded or replaced.

The ESP32 microcontroller serves as the central processing unit. It reads analog and digital signals from multiple sensors, processes them, and presents the data on a small 0.96-inch SSD1306 OLED display. It continuously collects data from connected sensors and provides two primary modes of feedback:

- Local feedback: Real-time display of readings and alerts on the OLED screen.
- Remote feedback: Transmission of encrypted sensor data to an MQTT broker for cloud visualization or application control.

Additionally, users can send commands either through the Serial Monitor (USB) or remotely via MQTT topics to adjust configuration thresholds or modify system behavior.

The design includes a persistent storage feature through EEPROM, ensuring user-defined settings are retained even after power loss.

2.2 Functional Workflow

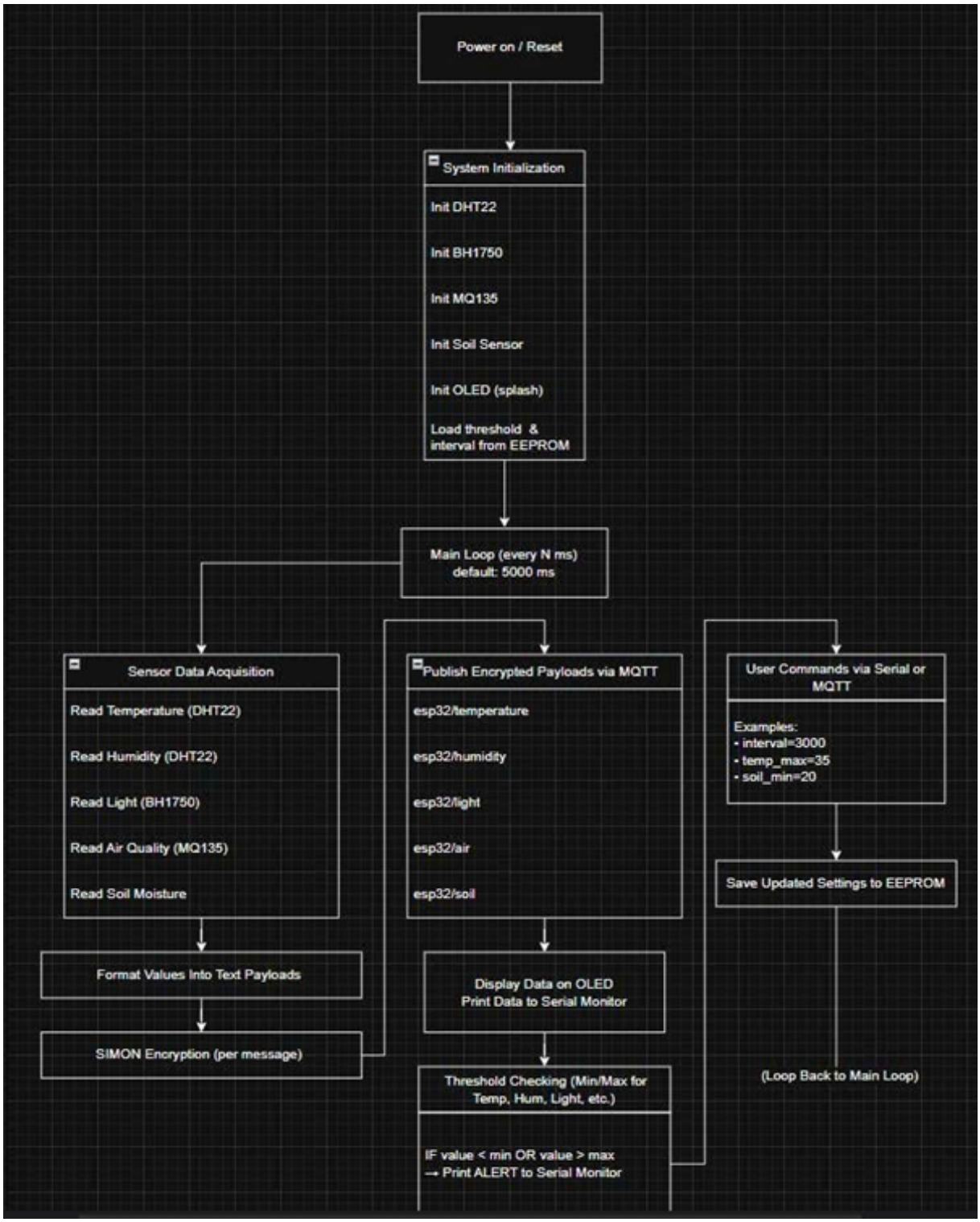

Figure 2.1: Functional Workflow Diagram

When the system powers on, it performs all necessary setup steps, including initializing the DHT22, BH1750, MQ135, and soil-moisture sensors; preparing the OLED display with a "Smart Plant Station" splash screen; and loading all previously saved threshold configurations from EEPROM. After initialization, the ESP32 continuously collects environmental data from every sensor at a configurable interval, which is set to 5000 ms by default. Each sensor reading is then formatted into a text string and encrypted using

the SIMON lightweight cipher before being published to the corresponding MQTT topics, such as esp32/temperature, esp32/humidity, esp32/light, esp32/air, and esp32/soil. All measured values are shown on the OLED display and output to the Serial Monitor, while the system simultaneously compares them against user-defined minimum and maximum thresholds; if any parameter falls outside its allowed range, an alert is immediately printed to the Serial Monitor. Users may adjust thresholds or change the data-collection interval at any time through serial commands or MQTT messages (e.g., interval=3000, temp_max=35), and all updates are instantly saved to EEPROM to ensure persistence. These saved configurations are automatically reloaded upon every reboot, allowing the Smart Plant Station to resume operation with the latest user settings.

Chapter 3

Hardware Design

3.1 Components

3.1.1 ESP32 development board / ESP32-WROOM-32 module

Getting Started with the ESP32 Development Board | Random Nerd Tutorials

The ESP32 serves as the central processing unit and controller for the entire project. It is a powerful 32-bit microcontroller developed by Espressif Systems and includes:

- Dual-core Tensilica LX6 processor (240 MHz)
- Integrated Wi-Fi (802.11 b/g/n)
- Bluetooth and BLE support
- 12-bit ADC inputs for analog sensors
- SPI, I^2C, and UART communication interfaces

The ESP32 was chosen due to its high performance, low power consumption, and rich peripheral support. It enables the system to collect sensor data, manage threshold logic, communicate with the OLED display, store user settings in EEPROM, and—in future upgrades—publish encrypted MQTT messages securely.

Two variants can be used:

- ESP32 DevKit v1 – Includes USB, regulator, and GPIO headers for easy prototyping
- ESP32-WROOM-32 Module—A smaller module meant for custom PCB integration
- Both versions share the same internal architecture, and the software code remains compatible between them.

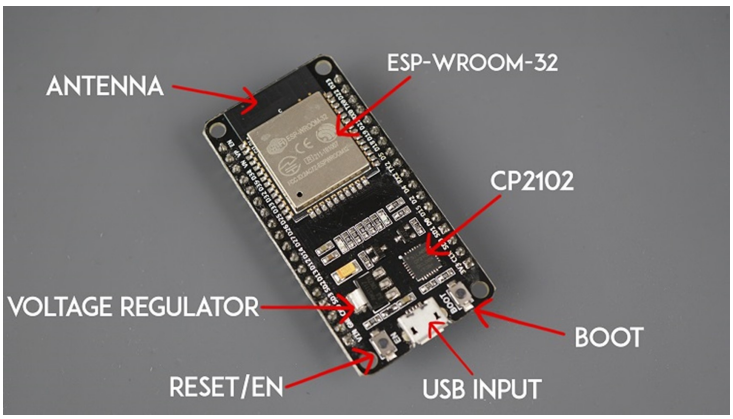

Figure 3.1: ESP32 Development Board

3.1.2 DHT22 temperature and humidity sensor

DHT22 Temperature-Humidity Sensor ...

The DHT22 (AM2302) is a digital environmental sensor that measures the following:

- Temperature: - 40°C to 80°C
- Humidity: 0% to 100% RH
- Accuracy: ± 2% RH and ± 0.5°C

The sensor simplifies integration without requiring analogue conversion by outputting calibrated digital data via a single GPIO pin (GPIO 4 in this case).

Justifications for selecting DHT22 over DHT11:

- Improved humidity resolution (0.1%)
- Greater accuracy
- Greater measuring range
- More stability in long-term monitoring

The sensor is crucial for identifying whether plants are subjected to harsh climatic conditions, low humidity, or heat exhaustion.

Figure 3.2: DHT22 Sensor

3.1.3 BH1750 light intensity sensor

The BH1750FVI is a digital lux sensor that measures ambient light using a high-sensitivity photodiode and an integrated analog-to-digital converter. It communicates with ESP32 using the I^2C protocol (SDA = GPIO 21, SCL = GPIO 22).

Characteristics:

- Measurement Range: 1 – 65,535 lux
- Output: Direct lux value (no manual calibration needed)
- Low noise and stable digital output
- Supports multiple measurement modes

The sensor plays a crucial role in determining whether plants receive sufficient sunlight or artificial light. Since light variation affects photosynthesis, the BH1750 allows the system to issue warnings when light levels are too low or excessively high.

Figure 3.3: BH1750 Sensor

3.1.4 MQ135 air quality sensor (Powered by 5V)

MQ135 Air Quality Sensor Module

The MQ135 gas sensor is designed for measuring air quality by detecting common gases such as

- Ammonia (NH_3)
- Carbon dioxide (CO_2)
- Benzene
- Alcohol
- Smoke
- Other organic compounds

The sensor outputs an analog voltage proportional to gas concentration, which is measured by the ESP32's ADC pin, GPIO 34.

Important characteristics:

- Power Requirement: 5V heater element
- Output Type: Analog
- High sensitivity to indoor pollutants
- Long warm-up period for optimal stability

In this system, MQ135 provides an approximate indicator of air quality near the plants. Although it is not a precise CO_2 meter, it helps identify general environmental issues such as stale air, smoke, or chemical exposure.

Because MQ135 requires 5V for proper operation, it is powered from the ESP32's 5V pin while its analog output is still safely read at 3.3V.

Figure 3.4: MQ135 Sensor

3.1.5 Soil moisture sensor (Analog type)

Soil sensor with Arduino in Analog mode ...

The analog soil moisture sensor evaluates the amount of water in soil by measuring its electrical resistance. The wetter the soil, the more conductive it becomes, producing a lower resistance and thus a lower analog voltage.

Connected to GPIO 35, the sensor provides:

- Indication of dry, moist, or oversaturated soil
- Real-time feedback for irrigation planning
- A basis for generating alerts (e.g., "Soil too dry")

Although these sensors are simple, they perform well for comparative measurements after calibration. Care should be taken to avoid long-term corrosion; the electrode should not be powered continuously unless necessary.

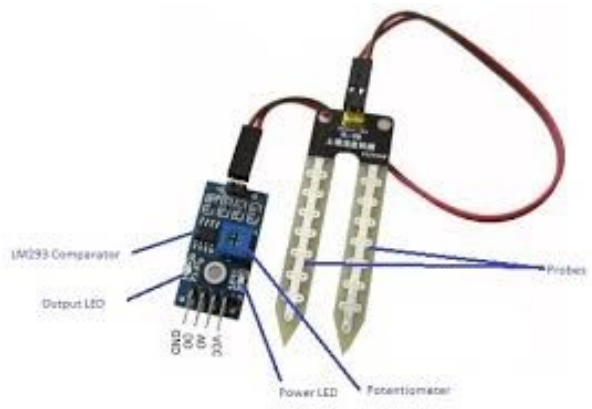

Figure 3.5: Soil Moisture Sensor

3.1.6 SSD1306 OLED display (128 × 64 pixels)

SSD1306 128x64 Pixel OLED Display ...

Real-time sensor data is shown visually on the 0.96-inch SSD1306 OLED display. It uses the same network as the BH1750 to communicate via I^2C.

Key characteristics include:

- Resolution: 128 × 64 monochrome pixels;
- Excellent contrast (OLED emits its own light);
- Low power consumption; and
- Integrated display buffer memory.

Temperature, humidity, light intensity, air quality, and soil moisture are all displayed. It can enable upcoming UI enhancements in addition to starting messages. Because users may view real-time data without a computer, the display improves system usability.

Figure 3.6: SSD1306 OLED Display

3.2 Circuit Design and Connection

The system operates on 3.3V logic and shares a common ground between all components, with only MQ135 being powered from 5V because its internal heater requires a stable 5V supply. The analog output remains within ESP32's 0–3.3V ADC range, making it safe to connect to GPIO34.

The OLED and BH1750 use the same I^2C bus, while analog sensors occupy ADC-capable GPIOs (34 and 35). This configuration ensures stable performance and minimizes signal interference.

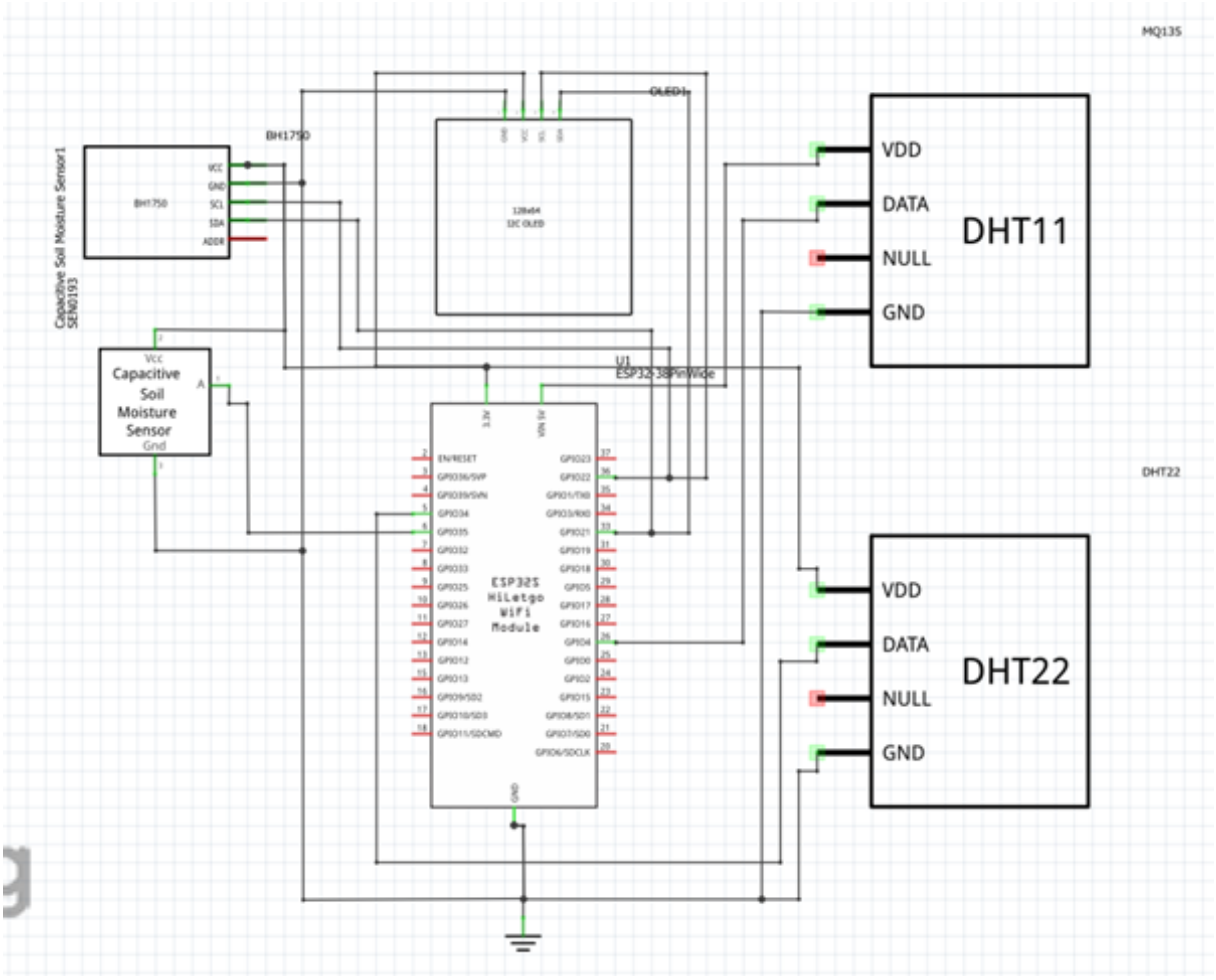

Figure 3.7: Circuit Design and Connection

Chapter 4

Encryption

Before transmitting wirelessly, data will be encrypted to ensure security and privacy. Therefore, our group decided to use the SIMON Lightweight Block Cipher to do this job.

What is SIMON?

SIMON is a family of lightweight block ciphers publicly released by the National Security Agency (NSA) in June 2013. Simon has been optimized for performance in hardware and resource-constrained devices such as: Embedded systems Microcontrollers (Arduino, STM32, MSP430, ESP32, AVR), RFID tags, sensor networks, IOT devices. SIMON has many advantages:

- Extremely small hardware footprint (as low as 1440 GE).
- Very high throughput.
- Minimal energy consumption.
- Only bitwise operations, making it easy to implement on nearly all embedded processors
- Highly suitable for ultra – low – resource devices.

Security properties:

- Differential attacks
- Linear cryptanalysis
- Impossible differential attacks
- Boomerang attacks
- Rotational cryptanalysis.

As of 2018, no successful attack on full-round Simon of any variant is known.

SIMON relies on:

- Bitwise AND
- Bitwise XOR
- Circular shifts/ rotation.

Next, overall structure of SIMON

SIMON is based on balanced Feistel network. Each round transforms the state (L_i, R_i) as follows:

$$L_{i+1} = R_i$$
$$R_{i+1} = L_i \oplus f(R_i) \oplus K_i$$

Note:

- L_i, R_i: the two halves of the data block.
- K_i: the round key for round i.
- $f(x)$: the nonlinear round function.

Parameters of SIMON.

SIMON has multiple blocks and key sizes, with each block is always split into two equal halves.

- For blocks sizes: 32,48,64,96,128 bits.
- For key sizes: 64,72,96,128,144,192,256 bits.

The SIMON f − function:

The nonlinear round function is:

$$f(x) = (x \lll 1 \ \& \ x \lll 8) \oplus (x \lll 2)$$

This uses only:

- Rotate-left by 1,8, and 2 bits.
- Bitwise AND
- XOR.

Encryption process

Here is how the encryption process takes place: Given plaintext block:

$$M = (L_0, R_0)$$

For each round:

$$R_{i+1} = L_i \oplus f(R_i) \oplus k_i$$
$$L_{i+1} = R_i$$

After the final round, we have:

$$C = (L_T, R_T)$$

C is the cipher text, with T = number of rounds. Because SIMON is Feistel-based, encryption requires only one direction of the f- function.

Decryption process:

Decryption simply reverses the Feistel process using round keys in reverse order.

$$L_i = R_{i+1}$$

$$R_i = L_{i+1} \oplus f(L_i) \oplus k_i$$

Starting from round T-1 down to 0.

SIMON Key schedule:

The key schedule expands the master key into round keys $K_0 \dots K_{T-1}$ by using:

- Rotate – right by 3.
- Rotate – right by 1.
- XOR with a constant.
- XOR with round counter.
- NSA -defined bit sequences called z-sequences.

So, our code implements a version of the SIMON encryption algorithm (SIMON128/128), Our code consists of 9 parts in total, which are:

1. 64-bit Rotation Functions
2. The SIMON f-Function (Core Round Function)
3. The 128-bit Master Key
4. Key Schedule — Generating 68 Round Keys
5. Encryption Function (SIMON128)
6. Decryption Function
7. Big-Endian Conversion Helpers
8. High-Level String Encryption Wrapper
9. High-Level String Decryption Wrapper.

Explanation:

1/ 64-bit Rotation Functions

Rotation operations are fundamental to the SIMON cipher. That's why this first part is so important. The SIMON f-function requires left rotations by 1, 2, and 8 bits. Rotations provide both non-linearity and diffusion.

- ROL64: rotate a 64-bit value left.
- ROR64: rotate right.

2/ The SIMON f-Function (Core Round Function)

The SIMON round function is:

$$f(x) = (x \lll 1 \ \& \ x \lll 8) \oplus (x \lll 2)$$

This is the main source of security in SIMON and our code simply applies this formula.

3/ The 128-bit Master Key

Our SIMON 128/128 code uses a 128 bit-key, stored as two 64-bit words. The code in this part creates the key. The reason we code this is because All SIMON variants split the key into m words, and in our case, m = 2, so the key is k0,k1.

4/ Key Schedule — Generating 68 Round Keys

SIMON128/128 requires exactly 68 round keys.so here is how we code: Initially, we have two 64-bit key words. Then there is a fixed constant used to prevent weak-key patterns. This is simply $2^{64} - 4$, as defined by SIMON. A predefined binary sequence (z_0) used for randomness This one (z0) is officially assigned for SIMON128/128. Here is the SIMON key generation formula:

$$k_i = k_{i-m} \oplus c \oplus z_j \oplus (k_{i-1} \ggg 3) \oplus (k_{i-1} \ggg 4)$$

With m = 2 Each time it will generate 2 round keys → A → B, then update A and B according to the above formula. Take each bit of Z and shift the round right by 3 and 4 bits to prevent weak keys. The same goes for final 4 round keys, so the total keys = 68, index from 0 to 67.

5/ Encryption Function (SIMON128)

Our code splits the 128-bit block into two 64-bit halves, then we apply the official formula for encrypting SIMON. (the formula is shown above). Finally, we have the cipher text.

6/ Decryption Function

For decrypting, it is exactly the inverse of the encryption process, just by reversing the round key order.

7/ Big-Endian Conversion Helpers

SIMON works with 64-bit integers, but external data is in bytes, so we need to convert byte to bits and bits to byte. Specifically, we convert 8 bytes → 64-bit big-endian in order to read bytes in big-endian order. Converting 64-bit → 8 bytes in order to store 64-bit value into big-endian bytes.

8/ High-Level String Encryption Wrapper

With this function, its jobs are to pad the string to 16 bytes, split into two 64-bit halves, call the SIMON encryption and finally outputs a 32-character hex string.

9/ High-Level String Decryption Wrapper

With this function, its jobs are to parse a 32-hex-character string, decrypts it, then converts the plaintext bytes back to characters and eventually removes padding spaces, output the text for users to read.

Chapter 5

Software Design

5.1 Libraries and Dependencies

The Smart Plant Monitoring System relies on several external software libraries to simplify hardware communication, sensor interfacing, data processing, and network connectivity. Each library provides specialized functionality that enables the ESP32 to operate efficiently and interact with peripherals. A detailed description of each dependency is provided below:

```
#include <WiFi.h>
#include <EEPROM.h>
#include <PubSubClient.h>
#include <Wire.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_GFX.h>
#include <Adafruit_Sensor.h>
#include <DHT.h>
#include <BH1750.h>
```

a. WiFi.h

This library provides the core functionality required for the ESP32 to initialize and manage Wi-Fi connections. It enables the device to connect to a local wireless network, obtain an IP address, and maintain network communication. The library also supports essential network operations such as reconnection handling, signal strength querying, and network configuration. It serves as the foundation for higher-level IoT protocols such as MQTT.

b. PubSubClient.h

PubSubClient is an MQTT client library that allows the ESP32 to publish and subscribe to topics on an MQTT broker. It handles packet formatting, message transmission, keepalive signaling, and callback processing. This library is crucial for cloud-based communication, enabling the system to send sensor data to remote servers and receive commands from external applications or dashboards.

c. Wire.h

The Wire library implements the I^2C (Inter-Integrated Circuit) communication protocol, which is essential for interfacing with the SSD1306 OLED display and the BH1750 light sensor. It manages clock and data signaling, device addressing, and synchronous communication between the ESP32 and multiple I^2C peripherals on the same bus.

d. Adafruit_SSD1306.h and Adafruit_GFX.h

These two libraries work together to operate the 0.96-inch SSD1306 OLED display.

`Adafruit_SSD1306.h` handles low-level communication with the display controller, including display initialization, buffer management, and pixel-level control.

`Adafruit_GFX.h` provides a comprehensive graphics framework offering functions for drawing shapes, rendering text, controlling fonts, and managing screen layout. Combined, they enable the system to present a clear and readable visual output of sensor data.

e. DHT.h

This library provides functions to read temperature and humidity data from the DHT22 sensor. It manages signal timing, error detection, and data decoding, converting the sensor's digital waveform into usable numerical values. The library abstracts the sensor's strict timing requirements, ensuring stable and reliable readings.

f. BH1750.h

The BH1750 library is used to interface with the BH1750 digital light intensity sensor over I^2C. It includes routines for changing measurement modes, reading lux values, and ensuring accurate sampling under different lighting conditions. The library simplifies the process of retrieving calibrated, high-resolution light measurements.

g. EEPROM.h

EEPROM.h provides access to the ESP32's emulated EEPROM storage, allowing the system to save configuration parameters such as threshold values and update intervals. It offers convenient functions for writing structured data, committing changes to flash memory, and retrieving stored values during startup. This ensures that user-defined settings persist even after the device is powered off or reset.

h. Adafruit_Sensor.h

This is a unified sensor abstraction layer used by several Adafruit libraries. It standardizes sensor data structures and measurement functions, providing a consistent representation of readings such as temperature, humidity, or light intensity. It enhances code modularity and compatibility across multiple sensor types.

5.2 Code structure

a. Wi-Fi and MQTT Configuration

This block defines the Wi-Fi network name, password, and MQTT broker address. A `WiFiClient` object handles TCP/IP communication, while `PubSubClient` manages MQTT messaging. These settings allow the ESP32 to connect to the internet and communicate with a remote MQTT broker.

b. OLED Display Initialization

The OLED screen resolution is defined as 128×64 pixels. An `Adafruit_SSD1306` display object is created, using the I^2C bus as the communication channel. This object is responsible for rendering all text and sensor values on the OLED screen.

c. Sensor Definition

- The DHT22 sensor is assigned to GPIO 4.
- A BH1750 light sensor object is created for I^2C communication.
- The MQ135 air quality sensor is connected to ADC pin 34.
- The soil moisture sensor is connected to ADC pin 35.

These sensors provide all environmental data for the plant monitoring system.

d. Threshold Structure

A struct `Thresholds` defines minimum and maximum values for all measured parameters. This includes temperature, humidity, light intensity, air quality, soil moisture, and update interval. An initial threshold object (th) is created with default values for safe plant conditions. These thresholds can later be modified by the user through Serial or MQTT commands.

e. EEPROM Storage

The program allocates EEPROM memory equal to the size of the `Thresholds` structure.

- `saveThresholds()` writes the threshold values into non-volatile memory.
- `loadThresholds()` retrieves previously saved configuration data after a restart.

This ensures user settings persist even after a power loss.

f. SIMON Encryption

This section implements the 128-bit SIMON lightweight block cipher for ESP32.

- `f64()` function defines the SIMON nonlinear round function using bit rotations.
- `simon128_key_schedule()` expands the 128-bit key into 68 round keys.
- `simon128_encrypt()` runs 68 rounds to produce the ciphertext block.
- `simon128_decrypt()` runs the rounds in reverse to recover plaintext.

- Helper functions convert between bytes and 64-bit words in big-endian format.
- `simon128EncryptString()` turns a 16-byte string into a 32-hex-character encrypted output.
- `simon128DecryptString()` converts the hex ciphertext back into readable text.

This module provides fast, lightweight encryption suitable for protecting MQTT sensor data.

g. Wi-Fi Setup

`setup_wifi()` function connects the ESP32 to the Wi-Fi network. It waits until the connection is successful and prints the assigned IP address. This step is required before any MQTT communication can occur.

h. MQTT Connection

The `reconnect()` function attempts to connect to the MQTT broker if disconnected. Once connected, the ESP32 subscribes to the topic "esp32/commands_enc". This allows the device to receive encrypted control commands.

i. Command Handling

`handleCommand()` processes incoming text commands. It checks for keywords such as `temp_max=`, `hum_min=`, or `interval=`. When a setting changes, it saves the new threshold to EEPROM and publishes an update alert. This function supports configuration through Serial or MQTT.

j. MQTT Callback

`callback()` receives encrypted MQTT messages from the topic "esp32/commands_enc". It decrypts the message using SIMON. The decrypted command is passed directly to `handleCommand()`. This enables secure remote configuration of the system.

k. Setup

`setup()` function initializes Serial communication, I^2C bus, EEPROM, Wi-Fi, and MQTT. It loads saved thresholds, starts all sensors, and initializes the OLED display. A startup message is shown on the screen, and command instructions are printed for the user.

l. Main Loop

`loop()` maintains the MQTT connection and processes any Serial commands. A timer checks if it's time to read sensors based on the user-defined interval. All sensors are read: temperature, humidity, light, air quality, and soil moisture.

m. Encryption, Decryption, and Publishing

Each sensor value is converted to a string and encrypted using SIMON. Encrypted values are published over MQTT to topics such as "esp32/temperature" and "esp32/light". For debugging, the program also decrypts its own messages to verify correctness.

n. OLED Display

Sensor readings are printed to the OLED display. The screen is refreshed every cycle to show up-to-date environmental data. This provides immediate visual feedback without needing a PC.

o. Alert and Threshold Check

Each reading is compared with its corresponding threshold. If a value is too high or too low, an alert string is generated. Alerts are printed to Serial and published to the MQTT topic "esp32/alerts". This enables real-time warning notifications.

Chapter 6

App

6.1 Overview of the Application

Our PC application provides a simple and clean dashboard that displays real-time sensor values. It connects to the ESP32 through MQTT and uses the same SIMON-128/128 encryption algorithm, ensuring secure communication between the PC and the device. With this application, users can:

- Monitor all sensors in real time
- See whether each value is normal or outside the safe range
- Receive warning messages directly from the ESP32
- Adjust threshold values for automation
- Send encrypted commands to the ESP32

This app essentially acts as the main "control center" of the entire system.

6.2 Application Structure

The program is divided into several main components:

a. SIMON-128 Encryption Module

We implemented the full SIMON-128/128 block cipher in Python to match the encryption on the ESP32. This module performs:

- Key expansion
- Block encryption and decryption
- String-to-ciphertext conversion
- Ciphertext-to-plaintext recovery

This ensures that all commands sent to the ESP32 remain encrypted and protected.

b. MQTT Communication

The application uses paho-mqtt to communicate with the HiveMQ broker. It subscribes to the following encrypted sensor topics:

- esp32/temperature
- esp32/humidity
- esp32/light
- esp32/air
- esp32/soil
- esp32/alerts* — alert messages

When the ESP32 sends sensor readings, the app decrypts them before displaying them. When users adjust thresholds or send commands, the app encrypts the commands and publishes them to: esp32/commands_enc This keeps the communication secure on both sides.

c. Graphical User Interface (GUI)

We built the GUI using Tkinter because it is lightweight and works well for desktop applications. The interface includes:

- Sensor Cards
- Each card shows:
  - Sensor name
  - Live value
  - Status (Good/Bad)
  - An emoji indicator
  - A color-coded background
- Values automatically change to "Bad" when they exceed the thresholds.
- Warning Panel
- At the bottom of the window, we added a warning panel that works like a simple chat box.
- Whenever the ESP32 detects abnormal sensor readings, it sends a warning that appears here.
- Menu Panel

The menu allows users to:

- Enter commands such as temp_min=25
- Reset all thresholds
- Show current threshold values

This makes controlling the ESP32 convenient from the PC.

6.3 Threshold Storage

We store threshold values in a local file called thresholds.json. This means users do not need to re-enter their settings every time the app starts. The system automatically loads the saved thresholds on startup, improving usability.

6.4 Real-Time Sensor Handling

When encrypted sensor values arrive from the ESP32, the application:

- Receives the MQTT message
- Decrypts the data using SIMON-128
- Extracts the numerical value
- Updates the corresponding sensor card
- Compares the value with the thresholds
- Displays a warning if needed

This process ensures that the readings shown on the screen always reflect the actual sensor measurements in real time.

Chapter 7

Applications and Future Development of the Smart Plant Monitoring System

7.1 Practical Applications

This system can be used in a number of real-world situations, starting with home gardening automation, which enables people to continuously monitor the temperature, humidity, sunlight intensity, air quality, and soil moisture surrounding their houseplants. Users receive encrypted MQTT alerts whenever any parameter deviates from the ideal range, which helps improve plant health and lessens the effort needed for novice or busy plant owners. The multi-sensor architecture offers real-time feedback that helps maintain stable temperature and humidity levels—conditions necessary for successful controlled agriculture—in more controlled settings like indoor hydroponics, tiny greenhouses, grow tents, or mini-farm setups. Because encrypted MQTT data can be incorporated into cloud dashboards, the technology can even be used in remote nurseries or dispersed small farms with less direct oversight. It is helpful in schools, offices, labs, and other interior locations where maintaining adequate ventilation and illumination is crucial, since it has MQ135 and BH1750 sensors, which enable it to evaluate indoor air quality and lighting conditions in addition to plant-focused applications.

7.2 Potential Future Enhancements

By adding relays, pumps, or solenoid valves, the system can be expanded with automated watering and actuator control, allowing it to react directly to soil-moisture readings and water plants automatically. This effectively turns the project from a straightforward monitoring solution into a fully autonomous plant-care system. Furthermore, adding solar panels and sleep-mode power optimizations would enable the gadget to function autonomously without the need for external power sources, making it appropriate for outdoor or isolated settings.

7.3 Long-Term Vision

This project can grow into a fully automatic smart-farming system.

- It can control pumps, lights, and fans by itself and react instantly to sensor changes to protect plants without human input.
- Data can be uploaded to the cloud for long-term storage, trend analysis, and accurate predictions of plant needs.
- Multiple ESP32 nodes can be added to cover large farms, all managed by a central server to create a complete smart-farming network.
- Security can be improved with encrypted MQTT, passwords, and certificates to prevent unauthorized access.
- The system can run on low power using solar panels and deep-sleep mode for long-term remote operation.
- Ultimately, this project can evolve into a full smart agriculture platform that saves water, improves plant health, increases yield, and supports sustainable farming.

Chapter 8

Conclusion

In this project, we successfully built a smart plant monitoring system using the ESP32, various sensors, MQTT communication, and the SIMON encryption algorithm. The system can read important environmental values such as temperature, humidity, light, air quality, and soil moisture, then send them securely to a computer. We also added an OLED screen, EEPROM storage, and remote commands from a Python program, which made the project more practical and easy to control.

Overall, the system works reliably and meets the goals we set at the beginning. Through this project, we learned how to connect hardware with software, use IoT communication, and apply real encryption in a real system. In the future, the project can be expanded with automatic watering, mobile app notifications, cloud dashboards, or multiple ESP32 nodes to create a more complete smart-farming system.

Chapter 9

Code and Demo

Code:
https://github.com/tuongnguyen274-tech/Do-An-TKLL---Nhom-5.git

Video demo:
https://drive.google.com/drive/folders/1pxAQq_FwaYdVVHnoRaM5L6A9y8KmDsJ-?usp=sharing